CPSC 24500 — Object-Oriented Programming

Week 1
Developer Toolkit and Python Basics

Lecture Notes

---

1 Learning Outcomes

By the end of this week, you should be able to:

1. Navigate the file system using terminal commands.
2. Create, move, copy, and delete files and directories from the command line.
3. Explain what version control is and why it matters.
4. Use core Git commands to track changes in a project.
5. Create and manage repositories on GitHub.
6. Work with Git branches to manage parallel development.
7. Install and verify Python 3 and pip.
8. Run Python programs from the command line and use the Python REPL.
9. Declare variables and use Python's core data types: `int`, `float`, `str`, and `bool`.
10. Read user input with `input()` and display output with `print()`.
11. Format strings using f-strings.

2 Terminal Basics

The terminal (also called the command line or shell) is a text-based interface to your computer. Instead of clicking through folders in a file manager, you type commands. This is faster, scriptable, and essential for working with Git and running Python programs.

2.1 Opening a Terminal

- **macOS**: Open `Terminal.app` (in Applications → Utilities) or use Spotlight (`Cmd+Space`, type "Terminal").
- **Windows (WSL)**: Open **Ubuntu** from the Start menu (after installing WSL — see the setup section below).
- **Windows (PowerShell)**: Right-click the Start button and select **Windows Terminal** or **PowerShell**. PowerShell works for Git and Python, but some commands differ from Linux/macOS.
- **Linux**: Open your distribution's terminal emulator (often `Ctrl+Alt+T`).

- **VS Code**: Open the integrated terminal with `Ctrl+‘` (backtick).

Tip

Windows users: We recommend WSL (covered in the setup section below) because it gives you a real Linux terminal. All examples in this course use Linux/macOS commands. If you use PowerShell instead, most commands are the same, but a few differ — we note those differences where they come up.

2.2 PowerShell vs. Linux/macOS Commands

Most terminal commands are the same across platforms, but a few differ:

Action	Linux/macOS/WSL	PowerShell
Print working directory	`pwd`	`pwd` (same)
List files	`ls`	`ls` or `dir`
Create empty file	`touch file.py`	`New-Item file.py`
View file contents	`cat file.py`	`cat file.py` or `Get-Content file.py`
Clear screen	`clear`	`cls` or `clear`
Activate venv	`source .venv/bin/activate`	`.venv\Scripts\activate`

2.3 Navigation Commands

Command	What It Does
`pwd`	Print working directory — shows where you are
`ls`	List files and folders in the current directory
`ls -la`	List all files including hidden ones, with details
`cd dirname`	Change directory into `dirname`
`cd ..`	Move up one directory
`cd ~`	Move to your home directory

Tip

Run `pwd` frequently when you are getting comfortable with the terminal. The most common beginner mistake is running commands in the wrong directory.

2.4 File and Directory Operations

Command	What It Does
`mkdir dirname`	Create a new directory
`touch filename`	Create an empty file (macOS/Linux)
`cat filename`	Display the contents of a file
`cp source dest`	Copy a file
`mv source dest`	Move or rename a file
`rm filename`	Delete a file (permanent — no recycle bin)
`rm -r dirname`	Delete a directory and everything inside it

Important

The `rm` command permanently deletes files. There is no undo and no recycle bin. Always double-check before running it.

2.5 Terminal Example

```
pwd
mkdir my_project
cd my_project
touch main.py
ls
cat main.py
```

This creates a new directory, moves into it, creates an empty Python file, lists the directory contents, and displays the (empty) file.

3 Version Control with Git

3.1 What Is Version Control?

Version control is a system that records changes to files over time so you can recall specific versions later. Think of it as unlimited undo with labels. Instead of saving files as `report_v1.py`, `report_v2_FINAL.py`, `report_v2_FINAL_fixed.py`, you save snapshots called **commits** with meaningful messages.

3.2 What Is Git?

Git is the most widely used version control system in the world. It is:

- **Distributed**: every developer has a full copy of the project history.
- **Fast**: operations happen locally on your machine.
- **Free and open source**: available on every platform.

3.3 Installing Git

- **macOS**: `xcode-select -install` (includes Git), or install via `brew install git`.
- **Windows**: Download Git for Windows from `https://git-scm.com`. This includes Git Bash.
- **Linux**: `sudo apt install git` (Debian/Ubuntu) or `sudo dnf install git` (Fedora).

Verify the installation:

```
git --version
```

3.4 First-Time Git Configuration

Before making your first commit, tell Git who you are:

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

These values appear in every commit you make.

3.5 The Git Workflow

Git has four key locations for your code:

1. **Working Directory**: the files on your disk that you edit.
2. **Staging Area**: a holding zone for changes you want in the next commit.
3. **Local Repository**: the full history of commits on your machine.
4. **Remote Repository**: a copy of the repo hosted online (e.g., GitHub).

The basic flow is:

$$\texttt{edit files} \longrightarrow \texttt{staging area} \longrightarrow \texttt{local repo} \longrightarrow \texttt{remote repo}$$

3.6 Core Git Commands

3.6.1 Creating a Repository

```
mkdir my_project
cd my_project
git init
```

`git init` creates a hidden `.git/` directory that stores the entire history of the project.

3.6.2 Staging and Committing

```
git add main.py
git commit -m "Add main script"
```

- `git add` moves files to the staging area.
- `git commit -m "message"` saves a snapshot with a descriptive message.
- Use `git add .` to stage all changed files at once.

Note

Editing a file does not automatically stage it. You must run `git add` before `git commit`. These are separate steps so you can choose exactly what goes into each commit.

3.6.3 Checking Status and History

```
git status
git log --oneline
git diff
```

- `git status` shows what has changed since the last commit.
- `git log -oneline` shows the commit history, one line per commit.
- `git diff` shows the exact lines that changed.

Tip

Run `git status` constantly, especially when you are learning. It tells you exactly what state your repository is in.

3.7 The .gitignore File

Some files should never be committed: compiled bytecode, virtual environments, editor settings, operating system files, and secrets. Create a `.gitignore` file in the root of your repository to tell Git to ignore them:

```
__pycache__/
*.pyc
.venv/
.env
.DS_Store
```

.gitignore

3.8 Branches

A branch is an independent line of development. The default branch is called `main`. You can create new branches to work on features without affecting the main codebase:

```
git branch feature-login
git checkout feature-login
```

Or create and switch in one step:

```
git checkout -b feature-login
```

When the feature is ready, you merge it back into `main`:

```
git checkout main
git merge feature-login
```

4 GitHub

4.1 What Is GitHub?

GitHub is a hosting service for Git repositories. Git is the tool that runs on your machine; GitHub is a website where you can store your repositories online, share them, and collaborate with others.

4.2 Creating a GitHub Account

1. Go to `https://github.com` and sign up for a free account.
2. Use a professional username — this will be visible to employers.
3. Verify your email address.

4.3 Creating a Repository on GitHub

1. Click the `+` button in the top-right corner and select **New repository**.
2. Name it (e.g., `lastname_cpsc24500`).
3. Set it to **Public**.
4. Do **not** add a README, .gitignore, or license (you will create these locally).
5. Click **Create repository**.

4.4 Connecting a Local Repo to GitHub

After creating the repository on GitHub, link your local repository to it:

```
git remote add origin https://github.com/yourusername/yourrepo.git
git push -u origin main
```

- `git remote add origin <url>` tells Git where the remote repository lives.
- `git push -u origin main` uploads your commits. The `-u` flag sets the upstream so future pushes can simply be `git push`.

4.5 Cloning an Existing Repository

To download someone else's repository (or your own from another machine):

```
git clone https://github.com/username/repo.git
```

This creates a local copy of the entire repository, including all of its history.

4.6 Pulling Changes

If the remote has new commits (e.g., from a collaborator), download them with:

```
git pull
```

4.7 GitHub Desktop (Optional GUI Alternative)

If you prefer a visual interface for Git instead of the command line, GitHub Desktop is a free application that handles Git operations through a graphical interface.

4.7.1 Installing GitHub Desktop

1. Go to `https://desktop.github.com` and click **Download**.
2. Run the installer.
3. When it opens, click **Sign in to GitHub.com** and log in with your GitHub account.
4. It will ask to configure Git. Use the same name and email you used for `git config` earlier.

4.7.2 Cloning a Repository with GitHub Desktop

1. Click **File** → **Clone Repository**.
2. Select your repository from the list (or paste the URL).
3. Choose where to save it on your computer and click **Clone**.

4.7.3 Making Commits with GitHub Desktop

1. Edit files in VS Code (or any editor).
2. Switch to GitHub Desktop — it will show your changes automatically.
3. In the bottom-left, type a commit message (e.g., "Add welcome script").
4. Click **Commit to main**.
5. Click **Push origin** at the top to upload to GitHub.

Note

GitHub Desktop is a convenient option, but you should still learn the command-line Git commands. Many professional workflows, job interviews, and CI/CD systems require command-line Git. Use GitHub Desktop as training wheels, not a permanent replacement.

4.8 Adding a Collaborator on GitHub

For this course, you need to add the instructor as a collaborator so they can access your repository for grading.

1. Go to your repository on `https://github.com`.
2. Click **Settings** (the gear icon near the top of the page).
3. In the left sidebar, click **Collaborators** (under "Access").
4. GitHub may ask you to confirm your password.
5. Click **Add people**.
6. Type the instructor's GitHub username: **adibenedetto117**

7. Click **Add adibenedetto117 to this repository**.

The instructor will receive an invitation and accept it. Do this as soon as you create your repository — do not wait until the deadline.

5 Setting Up Your Development Environment

This section walks you through every step of setting up the tools you need for this course. Follow the instructions for your operating system. If you get stuck, post on the discussion board — include a screenshot of the error.

5.1 Windows Users: Installing WSL

WSL (Windows Subsystem for Linux) gives you a real Linux terminal inside Windows. This is the recommended setup for this course because it ensures your terminal commands, Python, and Git all work exactly as they do on macOS and Linux.

5.1.1 Step 1: Enable WSL

Open **PowerShell as Administrator** (right-click the Start button and select "Windows Terminal (Admin)" or "PowerShell (Admin)") and run:

```
wsl --install
```

This installs WSL 2 with Ubuntu by default. When it finishes, **restart your computer**.

5.1.2 Step 2: Set Up Ubuntu

After restarting, Ubuntu will open automatically (or search for "Ubuntu" in the Start menu). It will ask you to create a username and password. This is your Linux username, not your Windows username.

Important

When you type your password, nothing will appear on screen — not even dots. This is normal. Just type your password and press Enter.

5.1.3 Step 3: Update Ubuntu

Run these commands to make sure everything is up to date:

```
sudo apt update
sudo apt upgrade -y
```

`sudo` means "run as administrator." It will ask for the password you just created.

5.1.4 Step 4: Install Python and pip

Ubuntu comes with Python 3, but you need pip (the package manager) and venv:

```
sudo apt install python3 python3-pip python3-venv -y
```

Verify:

```
python3 --version
pip3 --version
```

You should see version numbers (e.g., Python 3.12.x). If you see "command not found," something went wrong — post on the discussion board.

5.1.5 Step 5: Install Git

```
sudo apt install git -y
git --version
```

5.1.6 Where Are My Files?

Your WSL home directory is at `/home/yourusername/`. You can access your Windows files from WSL at `/mnt/c/Users/YourWindowsUsername/`, but it is better to keep your course files inside the WSL filesystem for performance.

Tip

Create a folder for this course right away:

```
mkdir -p ~/cpsc24500
cd ~/cpsc24500
```

5.2 macOS Users: Setup

5.2.1 Step 1: Install Homebrew (Recommended)

Homebrew is a package manager that makes installing software easy. Open `Terminal.app` and run:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/
    install/HEAD/install.sh)"
```

Follow the instructions that appear. After installation, close and reopen your terminal.

5.2.2 Step 2: Install Python

```
brew install python3
python3 --version
pip3 --version
```

Alternatively, download from https://www.python.org/downloads/.

5.2.3 Step 3: Install Git

If you have Homebrew:

```
brew install git
```

Or install Xcode command-line tools, which include Git:

```
xcode-select --install
```

Verify:

```
git --version
```

5.3 Linux Users: Setup

Most Linux distributions come with Python 3 pre-installed. Install the extras:

```
sudo apt install python3 python3-pip python3-venv git -y
python3 --version
pip3 --version
git --version
```

If you are on Fedora, use `sudo dnf install` instead of `sudo apt install`.

5.4 Setting Up VS Code

Visual Studio Code (VS Code) is a free, lightweight editor with excellent Python support. This is the recommended editor for this course.

5.4.1 Step 1: Download and Install

Go to `https://code.visualstudio.com` and download the installer for your operating system. Run the installer with default settings.

5.4.2 Step 2: Install Extensions

Open VS Code. Click the Extensions icon on the left sidebar (it looks like four squares) or press `Ctrl+Shift+X`. Search for and install:

1. **Python** by Microsoft — syntax highlighting, linting, debugging.
2. **WSL** by Microsoft — **Windows users only**. This lets VS Code work inside your WSL Ubuntu environment.

5.4.3 Step 3: Connect VS Code to WSL (Windows Only)

1. Open VS Code.
2. Press `Ctrl+Shift+P` to open the Command Palette.

3. Type "WSL" and select **WSL: Connect to WSL**.
4. VS Code will reload. You should see "WSL: Ubuntu" in the bottom-left corner.
5. Now when you open a terminal in VS Code (`Ctrl+‘`), it will be a Linux terminal inside WSL.

Important

Windows users: Always open VS Code through WSL. If you open it normally, you will be using Windows paths and the Windows version of Python, which will cause problems. The easiest way: open your Ubuntu terminal, navigate to your project folder, and type `code .` to open VS Code in the current directory.

5.4.4 Step 4: Open the Integrated Terminal

Press `Ctrl+‘` (backtick, the key above Tab). This opens a terminal at the bottom of VS Code. You will use this terminal for running Python, Git commands, and everything else in this course.

5.5 Installing Python (Without WSL)

If you are on Windows and choose not to use WSL, download Python from https://www.python.org/downloads/. During installation:

1. Check the box that says **"Add Python to PATH"** — this is critical.
2. Click **Install Now**.
3. After installation, open a new PowerShell window and verify:

```
python --version
pip --version
```

Important

On Windows without WSL, the command is `python` (not `python3`) and `pip` (not `pip3`). The notes and assignments use `python3` because that works on macOS, Linux, and WSL. Substitute `python` if you are on Windows without WSL.

6 Virtual Environments and requirements.txt

A **virtual environment** is an isolated copy of Python for your project. It keeps your project's packages separate from the system Python and from other projects. Every assignment in this course should use a virtual environment.

6.1 Why Virtual Environments?

Imagine two projects: one needs version 1.0 of a library, the other needs version 2.0. If both use the system Python, they conflict. Virtual environments solve this by giving each project its own set of packages.

6.2 Creating a Virtual Environment

Navigate to your project directory and run:

```
python3 -m venv .venv
```

This creates a folder called `.venv/` inside your project. It contains a copy of Python and a place to install packages. The dot prefix makes it a hidden folder.

6.3 Activating the Virtual Environment

Before you can use the virtual environment, you must **activate** it:

```
source .venv/bin/activate
```

When activated, your terminal prompt will change to show `(.venv)` at the beginning:

```
(.venv) user@machine:~/my_project$
```

Important

On Windows without WSL, the activation command is different:

```
.venv\Scripts\activate
```

6.4 Installing Packages with pip

With your virtual environment activated, you can install packages using `pip`:

```
pip install requests
```

This installs the `requests` library only inside your virtual environment, not system-wide.

To see what packages are installed:

```
pip list
```

6.5 The requirements.txt File

A `requirements.txt` file lists all the packages your project needs. This lets anyone else set up the same environment with one command.

6.5.1 Creating requirements.txt

After installing the packages your project needs, generate the file:

```
pip freeze > requirements.txt
```

This writes every installed package and its version to the file. It looks like:

```
requests==2.31.0
urllib3==2.2.1
```

requirements.txt

If your project uses only the Python standard library (no external packages), the file will be empty. **That is fine** — create an empty `requirements.txt` anyway, because it tells others that no external packages are needed.

6.5.2 Installing from requirements.txt

When someone clones your project, they create a virtual environment and install from the file:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

The `-r` flag tells pip to read the file and install everything listed.

6.6 Deactivating the Virtual Environment

When you are done working, deactivate the virtual environment:

```
deactivate
```

The `(.venv)` prefix will disappear from your prompt.

6.7 The Complete Workflow

Every time you start working on an assignment:

1. `cd` into your project directory.
2. Activate the virtual environment: `source .venv/bin/activate`
3. Work on your code, install packages if needed.
4. When done, update requirements: `pip freeze > requirements.txt`
5. Commit your changes: `git add .  && git commit -m "description"`
6. Deactivate: `deactivate`

Note

Never commit the `.venv/` folder to Git. It is large and platform-specific. The `.gitignore` file from the previous section already excludes it. Anyone can recreate it from `requirements.txt`.

7 Running Python Programs

There are two ways to run Python code:

7.1 The REPL (Read-Eval-Print Loop)

The REPL is an interactive mode where you type one line at a time and see the result immediately:

```
python3
```

```
>>> 2 + 3
5
>>> "hello".upper()
'HELLO'
>>> exit()
```

The REPL is useful for quick experiments and testing ideas. Type `exit()` or press `Ctrl+D` to leave.

7.2 Running a Script

Save your code in a `.py` file and run it from the terminal:

```
python3 main.py
```

This is how you will run all assignments in this course. Make sure your virtual environment is activated before running.

8 Variables and Data Types

8.1 Variables

A variable is a name that refers to a value stored in memory. In Python, you create a variable by assigning a value to a name with the `=` operator:

```
name = "Alice"
age = 20
gpa = 3.85
is_enrolled = True
```

Python figures out the type automatically based on the value you assign. You do not need to declare the type explicitly (unlike languages such as Java or C++).

8.2 Naming Rules

- Variable names can contain letters, digits, and underscores.
- They must start with a letter or underscore (not a digit).
- They are case-sensitive: `Name` and `name` are different variables.

- Use `snake_case` by convention: `student_name`, not `studentName`.
- Avoid Python reserved words: `if`, `for`, `class`, `return`, etc.

8.3 Core Data Types

Python has four fundamental data types you will use constantly:

Type	Example	Description
`int`	`42`	Whole numbers (positive, negative, or zero)
`float`	`3.14`	Decimal numbers
`str`	`"hello"`	Text (sequences of characters)
`bool`	`True`	Logical values: `True` or `False`

8.4 Checking Types

Use the built-in `type()` function to check what type a value is:

```
age = 20
print(type(age))

price = 9.99
print(type(price))

name = "Alice"
print(type(name))

enrolled = True
print(type(enrolled))
```

Output:

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

8.5 Type Conversion

You can convert between types using built-in functions:

```
x = "42"
y = int(x)
z = float(x)

print(y)
print(z)
print(type(y))
print(type(z))
```

Output:

```
42
42.0
<class 'int'>
<class 'float'>
```

Important

Converting a non-numeric string to `int` or `float` will cause a `ValueError`. For example, `int("hello")` will crash your program. We will learn how to handle this with exceptions in a later week.

9 Basic Input and Output

9.1 The print() Function

`print()` displays text to the terminal:

```
print("Hello, world!")
print(42)
print(3.14)
print(True)
```

You can print multiple values separated by commas. Python adds a space between them automatically:

```
name = "Alice"
age = 20
print("Name:", name, "Age:", age)
```

Output:

```
Name: Alice Age: 20
```

9.2 The input() Function

`input()` pauses the program and waits for the user to type something. It always returns a **string**:

```
name = input("Enter your name: ")
print("Hello,", name)
```

Note

`input()` always returns a string, even if the user types a number. If you need a number, you must convert it:

```python
age_str = input("Enter your age: ")
age = int(age_str)
```

10 F-Strings and String Formatting

10.1 What Are F-Strings?

F-strings (formatted string literals) are the modern way to embed expressions inside strings. Prefix the string with `f` and put expressions inside curly braces:

```python
name = "Alice"
age = 20
print(f"My name is {name} and I am {age} years old.")
```

Output:

```
My name is Alice and I am 20 years old.
```

10.2 Expressions Inside F-Strings

You can put any valid Python expression inside the curly braces:

```python
x = 10
y = 3
print(f"{x} + {y} = {x + y}")
print(f"{x} / {y} = {x / y:.2f}")
```

Output:

```
10 + 3 = 13
10 / 3 = 3.33
```

The `:.2f` is a format specifier that rounds the result to two decimal places.

10.3 Common Format Specifiers

Specifier	Effect
`:.2f`	Two decimal places (e.g., `3.14`)
`:>10`	Right-align in a field 10 characters wide
`:<10`	Left-align in a field 10 characters wide
`:^10`	Center in a field 10 characters wide
`:,`	Add comma separators (e.g., `1,000,000`)

10.4 F-Strings vs. Other Methods

Python has older formatting methods (`%` formatting, `.format()`). In this course we use f-strings exclusively because they are the most readable and the current standard:

```
name = "Alice"

print(f"Hello, {name}!")

print("Hello, {}!".format(name))

print("Hello, %s!" % name)
```

All three produce the same output. Use the first one.

11 Putting It All Together

Here is a complete program that uses everything from this week:

```
from datetime import datetime

name = input("Enter your name: ")
major = input("Enter your major: ")
today = datetime.now().strftime("%B %d, %Y")

print()
print("=" * 40)
print(f"  Welcome, {name}!")
print(f"  Major: {major}")
print(f"  Date:  {today}")
print("=" * 40)
```

This program:

1. Imports `datetime` from the standard library.
2. Reads two strings from the user.
3. Formats the current date as a human-readable string.
4. Prints a banner using string repetition (`"=" * 40`) and f-strings.

12 Summary

This week you learned the foundational tools and concepts you will use every week for the rest of the course:

- The **terminal** is your primary interface for running programs and managing files.
- **Git** tracks every change you make, giving you unlimited undo with full history.
- **GitHub** stores your repositories online and enables collaboration.

- **Python** is the language we will use for the entire course. You now know how to install it, run scripts, and use the REPL.
- **Variables** store values. Python's core types are `int`, `float`, `str`, and `bool`.
- `print()` and `input()` handle basic I/O.
- **F-strings** are the standard way to format output in modern Python.